

Suche nach dem B-A-C-H Motiv
in gewählten Werken von J.S. Bach

Oskar Nitzschker

5. April 2026



Bach.

1 Einleitung

Das Konzept der Vertonung des eigenen Namens ist, bzw. war ein beliebtes Konzept unter Komponisten diverser Epochen. Die bekannteste Umsetzung ist das B-A-C-H Motiv von Johann Sebastian Bach. In dem letzten Contrapunktus Satz der Kunst der Fuge erscheint das Motiv als Subjekt der dritten Fugenthemas. Im Folgenden werde ich beschreiben wie ich den music21 Bachcorpus und MXL-Dateien nach dem B-A-C-H Motiv durchsucht habe.

2 Prozess

2.1 Datenbeziehung

Die Untersuchung des music21 Corpus nach allen Werken von J.S. Bach lässt sich in einer Zeile Code programmieren:

```
bachCorpus = m21.corpus.search('bach', 'composer')
```

Die Variable `bachCorpus` enthält eine Liste aller von `m21.corpus.search()` gefundener Werke im MXL Format. Die Suche nach externen XML-Dateien ist ebenfalls simpel:

```
Directory = 'KdF/'
MXLFiles = []
for file in os.listdir(Directory):
    if file.endswith('.mxl'):
        MXLFiles.append(file)
```

Hier wird der Ordner `KdF` nach allen Dateien mit der Dateinendung `.mxl` untersucht. Alle gefundenen Ergebnisse werden folgend der Liste `MXLFiles` angehängt.

2.2 Suchfunktion

Sämtliche gesammelten `.mxl` Dateien werden iterativ der Funktion `findMotivFromMXL` übergeben. Diese Funktion beinhaltet zwei Unterfunktionen:

- `generateFlatScore()`
- `findMotiv()`

Für `generateFlatScore()` wird die `.mxl` Datei erst über die music21 Funktion `converter.parse()` zu einem von music21 verstehbaren Format konvertiert. `generateFlatScore()` gibt innerhalb einer Liste separat alle Stimmen aus. Die einzelnen Stimmen bestehen nur aus Noten und Pausen. Dementsprechend ist das ausgegebene Listenformat:

[[Noten & Pausen aus Stimme 1], [Noten & Pausen aus Stimme 2], [usw.]] Die zweite Funktion `findMotiv()` benötigt zwei Eingaben: 1. Die Noten & Stimmen Liste aus der vorherigen Funktion und 2. Das gesuchte Motiv. Dort anzumerken ist, dass das Motiv vorher in eine Intervalliste umgewandelt wurde, dazu im nächsten Unterabschnitt mehr. Die Funktion sieht folgendermaßen aus:

```
def findMotiv(flatScore, motiv):
    AmountOfParts = len(flatScore)
    lenMotiv = len(motiv[0])
    foundMotives = [], []
    for i in range(AmountOfParts):
        lenPart = len(flatScore[i])
        for j in range(lenPart):
            currentInterval = convertToIntervals(flatScore[i][j:j+lenMotiv])
```

```

if currentInterval == motiv[0]:
    foundMotives[0].append((i, (flatScore[i][j][1]+1)))
elif currentInterval == motiv[1]:
    foundMotives[1].append((i, (flatScore[i][j][1]+1)))
else:
    pass
return (len(foundMotives[0]), len(foundMotives[1]))

```

Die Funktion schaut sich ein Abschnitt (in der Länge des gesuchten Motivs) innerhalb einer Stimme an, vergleicht diesen mit dem Motiv und speichert den Index des Starttons, falls der Abschnitt mit dem Motiv übereinstimmt.

2.3 Motive & Intervalle

Um die Suche nach Transpositionen einfacher zu gestalten, benötigt `findMotiv()` das Motiv in der Form von Intervallen. Die Umrechnung in Intervallen findet für das Motiv in der Funktion `generateMotiveIntervals()` statt, während die untersuchten Abschnitte in der Funktion `convertToIntervals()` konvertiert werden. Beide Funktionen berechnen die Intervalle durch die Subtraktion der entsprechenden MIDI-Werte.

2.3.1 Anmerkung zu den Motiven

Mit einem Blick fällt auf, dass das gesuchte Motiv nur zwei Variationen enthält. Zufälligerweise sind die Intervalle der Original Motiv und der Krebs-Umkehrung, sowie der Umkehrung und der Krebsform die Gleichen. Demnach reduziert sich die zu suchende Anzahl von vier auf zwei.

3 Datenauswertung

Wie Anfangs beschrieben, werden sämtliche MXL-Dateien interaktiv untersucht. Die Ergebnisse werden in einer Liste der Form [Name, Anzahl Original, Anzahl Krebsform] (und ihre Equivalente) gespeichert. Mit Hilfe der Python Module `Pandas` und `MatPlotLib` werden diese Ergebnisse in einer .csv Datei gespeichert und abschließend dargestellt.

